COMPLETE SOURCE CODE

Smart Campus Navigation System | C11

This listing contains the complete C and header files copied from the project. Each filename is shown as a section heading. Syntax color is applied for readability; editable copies are in source_code/. test_driver.c is a separate executable entry point.

Application build: gcc -std=c11 -Wall -Wextra -Wpedantic -g main.c locations.c graph.c queue.c -o campus_navigation.exe

Test build: gcc -std=c11 -Wall -Wextra -Wpedantic -g test_driver.c locations.c graph.c queue.c -o test_driver.exe

main.c

```c
#include "graph.h"
#include "locations.h"

#include <ctype.h>
#include <errno.h>
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INPUT_LINE_CAPACITY 256

typedef enum {
 INPUT_RESULT_OK,
 INPUT_RESULT_INVALID,
 INPUT_RESULT_EOF
} InputResult;

static InputResult read_line(const char *prompt, char *buffer, size_t capacity)
{
 size_t length;

 if (buffer == NULL || capacity < 2) {
 return INPUT_RESULT_INVALID;
 }
 if (prompt != NULL) {
 fputs(prompt, stdout);
 fflush(stdout);
 }
 if (fgets(buffer, (int)capacity, stdin) == NULL) {
 return INPUT_RESULT_EOF;
 }

 length = strlen(buffer);
 if (length > 0 && buffer[length - 1] == '\n') {
 buffer[--length] = '\0';
 if (length > 0 && buffer[length - 1] == '\r') {
 buffer[--length] = '\0';
 }
 } else if (!feof(stdin)) {
 int character;
 while ((character = getchar()) != '\n' && character != EOF) {
 /* Discard the rest of this overlong line. */
 }
 puts("Input line is too long; please try again.");
 return INPUT_RESULT_INVALID;
 }

 return INPUT_RESULT_OK;
}

static InputResult read_integer(const char *prompt, int *value)
{
 char line[INPUT_LINE_CAPACITY];
 char *end;
 long parsed_value;
 InputResult result;

 if (value == NULL) {
 return INPUT_RESULT_INVALID;
 }

 result = read_line(prompt, line, sizeof(line));
 if (result != INPUT_RESULT_OK) {
 return result;
 }

 errno = 0;
 parsed_value = strtol(line, &end, 10);
 if (end == line || errno == ERANGE || parsed_value < INT_MIN ||
 parsed_value > INT_MAX) {
 puts("Invalid input; please enter a whole number.");
 return INPUT_RESULT_INVALID;
 }
 while (isspace((unsigned char)*end)) {
 end++;
```

```
 }
 if (*end != '\0') {
 puts("Invalid input; please enter a whole number without extra text.");
 return INPUT_RESULT_INVALID;
 }

 *value = (int)parsed_value;
 return INPUT_RESULT_OK;
}

static void print_location_reference(const LocationList *locations, int id)
{
 const Location *location = location_find_by_id(locations, id);

 if (location == NULL) {
 printf("ID %d", id);
 } else {
 printf("%s (ID %d)", location->name, location->id);
 }
}

static bool add_location_action(LocationList *locations, Graph *graph)
{
 int id;
 char name[INPUT_LINE_CAPACITY];
 InputResult input_result;
 GraphStatus graph_status;
 LocationStatus location_status;

 input_result = read_integer("Enter a positive location ID: ", &id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }

 input_result = read_line("Enter the location name (up to 63 characters): ",
 name, sizeof(name));
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }

 graph_status = graph_add_vertex(graph, id);
 if (graph_status != GRAPH_STATUS_OK) {
 printf("Location was not added: %s\n",
 graph_status_message(graph_status));
 return true;
 }

 location_status = location_list_add(locations, id, name);
 if (location_status != LOCATION_STATUS_OK) {
 GraphStatus rollback_status = graph_remove_vertex(graph, id);
 printf("Location was not added: %s\n",
 location_status_message(location_status));
 if (rollback_status != GRAPH_STATUS_OK) {
 puts("Internal consistency error: graph rollback failed.");
 }
 return true;
 }

 printf("Added %s (ID %d).\n",
 location_find_by_id(locations, id)->name, id);
 return true;
}

static bool search_location_action(const LocationList *locations)
{
 int search_type;
 InputResult input_result;
 const Location *location = NULL;

 input_result = read_integer("Search by 1) ID or 2) name: ", &search_type);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
```

```
 }

 if (search_type == 1) {
 int id;
 input_result = read_integer("Enter the location ID: ", &id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 location = location_find_by_id(locations, id);
 } else if (search_type == 2) {
 char name[INPUT_LINE_CAPACITY];
 input_result = read_line("Enter the location name: ", name,
 sizeof(name));
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 location = location_find_by_name(locations, name);
 } else {
 puts("Choose 1 to search by ID or 2 to search by name.");
 return true;
 }

 if (location == NULL) {
 puts("No matching location was found.");
 } else {
 printf("Found: %s (ID %d).\n", location->name, location->id);
 }
 return true;
}

static bool connect_locations_action(const LocationList *locations,
 Graph *graph)
{
 int id_a;
 int id_b;
 int metres;
 InputResult input_result;
 GraphStatus status;

 input_result = read_integer("Enter the first location ID: ", &id_a);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 input_result = read_integer("Enter the second location ID: ", &id_b);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 input_result = read_integer("Enter the walking distance in metres: ",
 &metres);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }

 status = graph_add_edge(graph, id_a, id_b, metres);
 if (status != GRAPH_STATUS_OK) {
 printf("Path was not added: %s\n", graph_status_message(status));
 return true;
 }

 printf("Added path: ");
 print_location_reference(locations, id_a);
 printf(" <-> ");
 print_location_reference(locations, id_b);
 printf(" (%d m).\n", metres);
 return true;
}
```

```c
static void display_paths(const LocationList *locations, const Graph *graph)
{
 size_t first_index;
 size_t vertex_count = graph_vertex_count(graph);
 size_t path_count = 0;

 for (first_index = 0; first_index < vertex_count; first_index++) {
 int first_id;
 size_t second_index;

 if (!graph_get_vertex_id_at(graph, first_index, &first_id)) {
 continue;
 }
 for (second_index = first_index + 1; second_index < vertex_count;
 second_index++) {
 int second_id;
 int metres;

 if (!graph_get_vertex_id_at(graph, second_index, &second_id) ||
 !graph_get_edge_weight(graph, first_id, second_id, &metres)) {
 continue;
 }

 if (path_count == 0) {
 puts("Walking paths:");
 }
 printf(" ");
 print_location_reference(locations, first_id);
 printf(" <-> ");
 print_location_reference(locations, second_id);
 printf(": %d m\n", metres);
 path_count++;
 }
 }

 if (path_count == 0) {
 puts("No walking paths have been added.");
 }
}

static bool display_paths_action(const LocationList *locations,
 const Graph *graph)
{
 display_paths(locations, graph);
 return true;
}

static bool shortest_route_action(const LocationList *locations,
 const Graph *graph)
{
 int start_id;
 int destination_id;
 int route[GRAPH_MAX_VERTICES];
 int total_metres;
 size_t route_length;
 size_t index;
 InputResult input_result;
 GraphStatus status;

 input_result = read_integer("Enter the starting location ID: ", &start_id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 input_result = read_integer("Enter the destination location ID: ",
 &destination_id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }

 status = graph_shortest_path(graph, start_id, destination_id, route,
 GRAPH_MAX_VERTICES, &route_length,
 &total_metres);
 if (status != GRAPH_STATUS_OK) {
 printf("Could not calculate route: %s\n",
```

```
 graph_status_message(status));
 return true;
 }

 printf("Shortest route: ");
 for (index = 0; index < route_length; index++) {
 if (index > 0) {
 printf(" -> ");
 }
 print_location_reference(locations, route[index]);
 }
 printf("\nTotal distance: %d m\n", total_metres);
 return true;
}

static bool bfs_action(const LocationList *locations, const Graph *graph)
{
 int start_id;
 int destination_id;
 int reachable[GRAPH_MAX_VERTICES];
 size_t reachable_count;
 size_t index;
 bool destination_reachable = false;
 InputResult input_result;
 GraphStatus status;

 input_result = read_integer("Enter the starting location ID: ", &start_id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }
 input_result = read_integer("Enter the destination location ID to check: ",
 &destination_id);
 if (input_result == INPUT_RESULT_EOF) {
 return false;
 }
 if (input_result != INPUT_RESULT_OK) {
 return true;
 }

 status = graph_bfs(graph, start_id, reachable, GRAPH_MAX_VERTICES,
 &reachable_count);
 if (status != GRAPH_STATUS_OK) {
 printf("Could not perform BFS: %s\n", graph_status_message(status));
 return true;
 }

 printf("Reachable locations in BFS order (%zu):\n", reachable_count);
 for (index = 0; index < reachable_count; index++) {
 printf(" ");
 print_location_reference(locations, reachable[index]);
 putchar('\n');
 if (reachable[index] == destination_id) {
 destination_reachable = true;
 }
 }

 if (!graph_has_vertex(graph, destination_id)) {
 printf("Destination ID %d is not in the graph.\n", destination_id);
 } else if (destination_reachable) {
 printf("Destination ");
 print_location_reference(locations, destination_id);
 printf(" is reachable from ");
 print_location_reference(locations, start_id);
 puts(".");
 } else {
 printf("Destination ");
 print_location_reference(locations, destination_id);
 printf(" is not reachable from ");
 print_location_reference(locations, start_id);
 puts(".");
 }
 return true;
}

static bool connectivity_action(const Graph *graph)
{
 bool connected;
 GraphStatus status = graph_is_connected(graph, &connected);
```

```
 if (status != GRAPH_STATUS_OK) {
 printf("Could not check connectivity: %s\n",
 graph_status_message(status));
 } else if (connected) {
 puts("The campus graph is connected.");
 } else {
 puts("The campus graph is not connected.");
 }
 return true;
}

static bool load_fictional_sample_data(LocationList *locations, Graph *graph)
{
 static const struct {
 int id;
 const char *name;
 } sample_locations[] = {
 {10, "Demo North Gate"},
 {73, "Demo Library"},
 {420, "Demo Cafeteria"},
 {501, "Demo Science Block"},
 {900, "Demo Sports Centre"}
 };
 static const struct {
 int id_a;
 int id_b;
 int metres;
 } sample_paths[] = {
 {10, 73, 120},
 {73, 420, 170},
 {10, 501, 260},
 {501, 420, 80},
 {73, 501, 150},
 {420, 900, 220},
 {501, 900, 350}
 };
 LocationList temporary_locations;
 Graph temporary_graph;
 const char *failure_message = NULL;
 size_t index;

 if (locations->count != 0 || graph_vertex_count(graph) != 0) {
 puts("Fictional sample data can only be loaded into an empty map.");
 return true;
 }

 location_list_init(&temporary_locations);
 graph_init(&temporary_graph);

 for (index = 0;
 index < sizeof(sample_locations) / sizeof(sample_locations[0]);
 index++) {
 GraphStatus graph_status =
 graph_add_vertex(&temporary_graph, sample_locations[index].id);
 LocationStatus location_status;

 if (graph_status != GRAPH_STATUS_OK) {
 failure_message = graph_status_message(graph_status);
 goto failure;
 }

 location_status = location_list_add(&temporary_locations,
 sample_locations[index].id,
 sample_locations[index].name);
 if (location_status != LOCATION_STATUS_OK) {
 (void)graph_remove_vertex(&temporary_graph,
 sample_locations[index].id);
 failure_message = location_status_message(location_status);
 goto failure;
 }
 }

 for (index = 0; index < sizeof(sample_paths) / sizeof(sample_paths[0]);
 index++) {
 GraphStatus status =
 graph_add_edge(&temporary_graph, sample_paths[index].id_a,
 sample_paths[index].id_b,
 sample_paths[index].metres);
 if (status != GRAPH_STATUS_OK) {
 failure_message = graph_status_message(status);
```

```
 goto failure;
 }
 }

 *locations = temporary_locations;
 *graph = temporary_graph;
 puts("Loaded five fictional demo locations and seven fictional paths.");
 puts("These labels and distances are development examples, not a real campus map.");
 return true;

failure:
 location_list_destroy(&temporary_locations);
 printf("Sample data was not loaded: %s\n", failure_message);
 return true;
}

static void display_menu(void)
{
 puts("\n=== Smart Campus Navigation System ===");
 puts("1. Add campus location");
 puts("2. Display all campus locations");
 puts("3. Search for a location");
 puts("4. Connect two locations");
 puts("5. Display connected paths");
 puts("6. Find the shortest route using Dijkstra");
 puts("7. Explore reachable locations using BFS");
 puts("8. Check graph connectivity");
 puts("9. Load fictional sample data");
 puts("10. Exit");
}

int main(void)
{
 LocationList locations;
 Graph graph;
 bool running = true;

 location_list_init(&locations);
 graph_init(&graph);

 while (running) {
 int choice;
 InputResult input_result;

 display_menu();
 input_result = read_integer("Choose an option: ", &choice);
 if (input_result == INPUT_RESULT_EOF) {
 puts("\nInput ended; exiting safely.");
 break;
 }
 if (input_result != INPUT_RESULT_OK) {
 puts("Please enter a valid menu number.");
 continue;
 }

 switch (choice) {
 case 1:
 running = add_location_action(&locations, &graph);
 break;
 case 2:
 location_list_display(&locations);
 break;
 case 3:
 running = search_location_action(&locations);
 break;
 case 4:
 running = connect_locations_action(&locations, &graph);
 break;
 case 5:
 (void)display_paths_action(&locations, &graph);
 break;
 case 6:
 running = shortest_route_action(&locations, &graph);
 break;
 case 7:
 running = bfs_action(&locations, &graph);
 break;
 case 8:
 (void)connectivity_action(&graph);
 break;
 case 9:
```

```
 (void)load_fictional_sample_data(&locations, &graph);
 break;
 case 10:
 running = false;
 puts("Goodbye.");
 break;
 default:
 puts("Invalid menu option. Choose a number from 1 to 10.");
 break;
 }

 if (!running && choice != 10) {
 puts("\nInput ended; exiting safely.");
 }
 }

 location_list_destroy(&locations);
 return EXIT_SUCCESS;
}
```

graph.c

```
#include "graph.h"

#include "queue.h"

#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

static bool find_vertex_index(const Graph *graph, int location_id,
 size_t *index_out)
{
 size_t index;

 if (graph == NULL || index_out == NULL ||
 graph->vertex_count > GRAPH_MAX_VERTICES) {
 return false;
 }

 for (index = 0; index < graph->vertex_count; index++) {
 if (graph->location_ids[index] == location_id) {
 *index_out = index;
 return true;
 }
 }

 return false;
}

void graph_init(Graph *graph)
{
 if (graph == NULL) {
 return;
 }

 memset(graph, 0, sizeof(*graph));
}

GraphStatus graph_add_vertex(Graph *graph, int location_id)
{
 size_t unused_index;

 if (graph == NULL) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 if (location_id <= 0) {
 return GRAPH_STATUS_INVALID_ID;
 }
 if (graph->vertex_count > GRAPH_MAX_VERTICES) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 if (find_vertex_index(graph, location_id, &unused_index)) {
 return GRAPH_STATUS_DUPLICATE_VERTEX;
 }
 if (graph->vertex_count == GRAPH_MAX_VERTICES) {
 return GRAPH_STATUS_CAPACITY_REACHED;
 }

 graph->location_ids[graph->vertex_count] = location_id;
 graph->vertex_count++;
 return GRAPH_STATUS_OK;
}

GraphStatus graph_remove_vertex(Graph *graph, int location_id)
{
 size_t removed_index;
 size_t old_count;
 size_t row;
 size_t column;

 if (graph == NULL) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 if (location_id <= 0) {
 return GRAPH_STATUS_INVALID_ID;
 }
 if (!find_vertex_index(graph, location_id, &removed_index)) {
```

```
 return GRAPH_STATUS_VERTEX_NOT_FOUND;
 }

 old_count = graph->vertex_count;
 for (row = removed_index; row + 1 < old_count; row++) {
 graph->location_ids[row] = graph->location_ids[row + 1];
 for (column = 0; column < old_count; column++) {
 graph->weights[row][column] = graph->weights[row + 1][column];
 }
 }

 for (row = 0; row + 1 < old_count; row++) {
 for (column = removed_index; column + 1 < old_count; column++) {
 graph->weights[row][column] = graph->weights[row][column + 1];
 }
 }

 graph->vertex_count--;
 graph->location_ids[graph->vertex_count] = 0;
 for (row = 0; row < old_count; row++) {
 graph->weights[graph->vertex_count][row] = 0;
 graph->weights[row][graph->vertex_count] = 0;
 }

 return GRAPH_STATUS_OK;
}

GraphStatus graph_add_edge(Graph *graph, int id_a, int id_b, int metres)
{
 size_t index_a;
 size_t index_b;

 if (graph == NULL) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 if (id_a <= 0 || id_b <= 0) {
 return GRAPH_STATUS_INVALID_ID;
 }
 if (metres <= 0) {
 return GRAPH_STATUS_INVALID_DISTANCE;
 }
 if (id_a == id_b) {
 return GRAPH_STATUS_SELF_CONNECTION;
 }
 if (!find_vertex_index(graph, id_a, &index_a) ||
 !find_vertex_index(graph, id_b, &index_b)) {
 return GRAPH_STATUS_VERTEX_NOT_FOUND;
 }
 if (graph->weights[index_a][index_b] != 0) {
 return GRAPH_STATUS_DUPLICATE_EDGE;
 }

 graph->weights[index_a][index_b] = metres;
 graph->weights[index_b][index_a] = metres;
 return GRAPH_STATUS_OK;
}

size_t graph_vertex_count(const Graph *graph)
{
 if (graph == NULL || graph->vertex_count > GRAPH_MAX_VERTICES) {
 return 0;
 }
 return graph->vertex_count;
}

bool graph_has_vertex(const Graph *graph, int location_id)
{
 size_t unused_index;

 return location_id > 0 &&
 find_vertex_index(graph, location_id, &unused_index);
}

bool graph_get_vertex_id_at(const Graph *graph, size_t index, int *location_id)
{
 if (graph == NULL || location_id == NULL ||
 index >= graph->vertex_count ||
 graph->vertex_count > GRAPH_MAX_VERTICES) {
 return false;
 }
```

```
 *location_id = graph->location_ids[index];
 return true;
}

bool graph_get_edge_weight(const Graph *graph, int id_a, int id_b,
 int *metres)
{
 size_t index_a;
 size_t index_b;

 if (graph == NULL || metres == NULL || id_a <= 0 || id_b <= 0 ||
 !find_vertex_index(graph, id_a, &index_a) ||
 !find_vertex_index(graph, id_b, &index_b)) {
 return false;
 }

 *metres = graph->weights[index_a][index_b];
 return *metres > 0;
}

GraphStatus graph_shortest_path(const Graph *graph,
 int start_id,
 int destination_id,
 int route_ids[],
 size_t route_capacity,
 size_t *route_length,
 int *total_metres)
{
 long long distances[GRAPH_MAX_VERTICES];
 size_t predecessors[GRAPH_MAX_VERTICES];
 size_t reverse_route[GRAPH_MAX_VERTICES];
 bool visited[GRAPH_MAX_VERTICES] = {false};
 size_t start_index;
 size_t destination_index;
 size_t route_size = 0;
 size_t index;

 if (graph == NULL || route_ids == NULL || route_length == NULL ||
 total_metres == NULL || graph->vertex_count > GRAPH_MAX_VERTICES) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 *route_length = 0;
 *total_metres = 0;

 if (start_id <= 0 || destination_id <= 0) {
 return GRAPH_STATUS_INVALID_ID;
 }
 if (!find_vertex_index(graph, start_id, &start_index) ||
 !find_vertex_index(graph, destination_id, &destination_index)) {
 return GRAPH_STATUS_VERTEX_NOT_FOUND;
 }

 for (index = 0; index < graph->vertex_count; index++) {
 distances[index] = LLONG_MAX;
 predecessors[index] = SIZE_MAX;
 }
 distances[start_index] = 0;

 for (index = 0; index < graph->vertex_count; index++) {
 size_t current = SIZE_MAX;
 size_t candidate;

 for (candidate = 0; candidate < graph->vertex_count; candidate++) {
 if (!visited[candidate] && distances[candidate] != LLONG_MAX &&
 (current == SIZE_MAX ||
 distances[candidate] < distances[current])) {
 current = candidate;
 }
 }

 if (current == SIZE_MAX) {
 break;
 }
 if (current == destination_index) {
 break;
 }
 visited[current] = true;

 for (candidate = 0; candidate < graph->vertex_count; candidate++) {
 int edge_weight = graph->weights[current][candidate];
 long long new_distance;
```

```
 if (edge_weight <= 0 || visited[candidate]) {
 continue;
 }
 if (distances[current] > LLONG_MAX - edge_weight) {
 return GRAPH_STATUS_DISTANCE_OVERFLOW;
 }

 new_distance = distances[current] + edge_weight;
 if (new_distance < distances[candidate]) {
 distances[candidate] = new_distance;
 predecessors[candidate] = current;
 }
 }
 }

 if (distances[destination_index] == LLONG_MAX) {
 return GRAPH_STATUS_NO_ROUTE;
 }
 if (distances[destination_index] > INT_MAX) {
 return GRAPH_STATUS_DISTANCE_OVERFLOW;
 }

 {
 size_t current = destination_index;
 while (current != SIZE_MAX && route_size < graph->vertex_count) {
 reverse_route[route_size++] = current;
 if (current == start_index) {
 break;
 }
 current = predecessors[current];
 }
 }

 if (route_size == 0 || reverse_route[route_size - 1] != start_index) {
 return GRAPH_STATUS_NO_ROUTE;
 }
 *route_length = route_size;
 if (route_capacity < route_size) {
 return GRAPH_STATUS_BUFFER_TOO_SMALL;
 }

 for (index = 0; index < route_size; index++) {
 route_ids[index] =
 graph->location_ids[reverse_route[route_size - index - 1]];
 }
 *total_metres = (int)distances[destination_index];
 return GRAPH_STATUS_OK;
}

GraphStatus graph_bfs(const Graph *graph,
 int start_id,
 int reachable_ids[],
 size_t reachable_capacity,
 size_t *reachable_count)
{
 bool visited[GRAPH_MAX_VERTICES] = {false};
 int order[GRAPH_MAX_VERTICES];
 size_t start_index;
 size_t order_count = 0;
 size_t index;
 Queue queue = {0};
 int current_value;

 if (graph == NULL || reachable_ids == NULL || reachable_count == NULL ||
 graph->vertex_count > GRAPH_MAX_VERTICES) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }
 *reachable_count = 0;

 if (start_id <= 0) {
 return GRAPH_STATUS_INVALID_ID;
 }
 if (!find_vertex_index(graph, start_id, &start_index)) {
 return GRAPH_STATUS_VERTEX_NOT_FOUND;
 }
 if (!queue_init(&queue, graph->vertex_count)) {
 return GRAPH_STATUS_OUT_OF_MEMORY;
 }

 visited[start_index] = true;
```

```
 if (!queue_enqueue(&queue, (int)start_index)) {
 queue_destroy(&queue);
 return GRAPH_STATUS_QUEUE_ERROR;
 }

 while (!queue_is_empty(&queue)) {
 size_t current;
 size_t neighbor;

 if (!queue_dequeue(&queue, &current_value)) {
 queue_destroy(&queue);
 return GRAPH_STATUS_QUEUE_ERROR;
 }
 current = (size_t)current_value;
 order[order_count++] = graph->location_ids[current];

 for (neighbor = 0; neighbor < graph->vertex_count; neighbor++) {
 if (graph->weights[current][neighbor] > 0 && !visited[neighbor]) {
 visited[neighbor] = true;
 if (!queue_enqueue(&queue, (int)neighbor)) {
 queue_destroy(&queue);
 return GRAPH_STATUS_QUEUE_ERROR;
 }
 }
 }
 }

 queue_destroy(&queue);
 *reachable_count = order_count;
 if (reachable_capacity < order_count) {
 return GRAPH_STATUS_BUFFER_TOO_SMALL;
 }

 for (index = 0; index < order_count; index++) {
 reachable_ids[index] = order[index];
 }
 return GRAPH_STATUS_OK;
}

GraphStatus graph_is_connected(const Graph *graph, bool *connected)
{
 int reachable_ids[GRAPH_MAX_VERTICES];
 size_t reachable_count = 0;
 GraphStatus status;

 if (graph == NULL || connected == NULL ||
 graph->vertex_count > GRAPH_MAX_VERTICES) {
 return GRAPH_STATUS_INVALID_ARGUMENT;
 }

 *connected = false;
 if (graph->vertex_count == 0) {
 return GRAPH_STATUS_OK;
 }

 status = graph_bfs(graph, graph->location_ids[0], reachable_ids,
 GRAPH_MAX_VERTICES, &reachable_count);
 if (status != GRAPH_STATUS_OK) {
 return status;
 }

 *connected = reachable_count == graph->vertex_count;
 return GRAPH_STATUS_OK;
}

const char *graph_status_message(GraphStatus status)
{
 switch (status) {
 case GRAPH_STATUS_OK:
 return "Operation completed successfully.";
 case GRAPH_STATUS_INVALID_ARGUMENT:
 return "Invalid graph argument or state.";
 case GRAPH_STATUS_INVALID_ID:
 return "Location IDs must be positive integers.";
 case GRAPH_STATUS_DUPLICATE_VERTEX:
 return "That location ID is already in the graph.";
 case GRAPH_STATUS_CAPACITY_REACHED:
 return "The graph has reached its 100-location limit.";
 case GRAPH_STATUS_VERTEX_NOT_FOUND:
 return "One or more location IDs are not in the graph.";
 case GRAPH_STATUS_INVALID_DISTANCE:
```

```
 return "Path distance must be a positive number of metres.";
 case GRAPH_STATUS_SELF_CONNECTION:
 return "A location cannot have a path to itself.";
 case GRAPH_STATUS_DUPLICATE_EDGE:
 return "A path already exists between those locations.";
 case GRAPH_STATUS_NO_ROUTE:
 return "No route connects those locations.";
 case GRAPH_STATUS_BUFFER_TOO_SMALL:
 return "The output array is too small for the result.";
 case GRAPH_STATUS_DISTANCE_OVERFLOW:
 return "The route distance exceeds the supported integer range.";
 case GRAPH_STATUS_OUT_OF_MEMORY:
 return "Memory allocation failed while creating the BFS queue.";
 case GRAPH_STATUS_QUEUE_ERROR:
 return "The BFS queue could not store or retrieve a vertex.";
 default:
 return "Unknown graph status.";
 }
}
```

graph.h

```h
#ifndef GRAPH_H
#define GRAPH_H

#include <stdbool.h>
#include <stddef.h>

#define GRAPH_MAX_VERTICES 100

typedef struct {
 int location_ids[GRAPH_MAX_VERTICES];
 int weights[GRAPH_MAX_VERTICES][GRAPH_MAX_VERTICES];
 size_t vertex_count;
} Graph;

typedef enum {
 GRAPH_STATUS_OK = 0,
 GRAPH_STATUS_INVALID_ARGUMENT,
 GRAPH_STATUS_INVALID_ID,
 GRAPH_STATUS_DUPLICATE_VERTEX,
 GRAPH_STATUS_CAPACITY_REACHED,
 GRAPH_STATUS_VERTEX_NOT_FOUND,
 GRAPH_STATUS_INVALID_DISTANCE,
 GRAPH_STATUS_SELF_CONNECTION,
 GRAPH_STATUS_DUPLICATE_EDGE,
 GRAPH_STATUS_NO_ROUTE,
 GRAPH_STATUS_BUFFER_TOO_SMALL,
 GRAPH_STATUS_DISTANCE_OVERFLOW,
 GRAPH_STATUS_OUT_OF_MEMORY,
 GRAPH_STATUS_QUEUE_ERROR
} GraphStatus;

void graph_init(Graph *graph);
GraphStatus graph_add_vertex(Graph *graph, int location_id);
GraphStatus graph_remove_vertex(Graph *graph, int location_id);
GraphStatus graph_add_edge(Graph *graph, int id_a, int id_b, int metres);
size_t graph_vertex_count(const Graph *graph);
bool graph_has_vertex(const Graph *graph, int location_id);
bool graph_get_vertex_id_at(const Graph *graph, size_t index, int *location_id);
bool graph_get_edge_weight(const Graph *graph, int id_a, int id_b,
 int *metres);
GraphStatus graph_shortest_path(const Graph *graph,
 int start_id,
 int destination_id,
 int route_ids[],
 size_t route_capacity,
 size_t *route_length,
 int *total_metres);
GraphStatus graph_bfs(const Graph *graph,
 int start_id,
 int reachable_ids[],
 size_t reachable_capacity,
 size_t *reachable_count);
GraphStatus graph_is_connected(const Graph *graph, bool *connected);
const char *graph_status_message(GraphStatus status);

#endif
```

locations.c

```c
#include "locations.h"

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Copy a trimmed name into a fixed-size buffer. */
static LocationStatus normalize_name(const char *name,
 char output[LOCATION_NAME_CAPACITY])
{
 const char *start;
 const char *end;
 size_t length;

 if (name == NULL || output == NULL) {
 return LOCATION_STATUS_INVALID_ARGUMENT;
 }

 start = name;
 while (*start != '\0' && isspace((unsigned char)*start)) {
 start++;
 }

 end = start + strlen(start);
 while (end > start && isspace((unsigned char)end[-1])) {
 end--;
 }

 length = (size_t)(end - start);
 if (length == 0) {
 return LOCATION_STATUS_EMPTY_NAME;
 }
 if (length > LOCATION_NAME_MAX) {
 return LOCATION_STATUS_NAME_TOO_LONG;
 }

 memcpy(output, start, length);
 output[length] = '\0';
 return LOCATION_STATUS_OK;
}

/* Compare two names without treating ASCII letter case as significant. */
static int names_equal_case_insensitive(const char *left, const char *right)
{
 while (*left != '\0' && *right != '\0') {
 unsigned char left_char = (unsigned char)*left;
 unsigned char right_char = (unsigned char)*right;

 if (tolower(left_char) != tolower(right_char)) {
 return 0;
 }
 left++;
 right++;
 }

 return *left == '\0' && *right == '\0';
}

void location_list_init(LocationList *list)
{
 if (list == NULL) {
 return;
 }

 list->head = NULL;
 list->tail = NULL;
 list->count = 0;
}

LocationStatus location_list_add(LocationList *list, int id, const char *name)
{
 char normalized_name[LOCATION_NAME_CAPACITY];
 LocationStatus status;
 Location *current;
 Location *new_location;
```

```
 if (list == NULL || name == NULL) {
 return LOCATION_STATUS_INVALID_ARGUMENT;
 }
 if (id <= 0) {
 return LOCATION_STATUS_INVALID_ID;
 }

 status = normalize_name(name, normalized_name);
 if (status != LOCATION_STATUS_OK) {
 return status;
 }

 for (current = list->head; current != NULL; current = current->next) {
 if (current->id == id) {
 return LOCATION_STATUS_DUPLICATE_ID;
 }
 if (names_equal_case_insensitive(current->name, normalized_name)) {
 return LOCATION_STATUS_DUPLICATE_NAME;
 }
 }

 new_location = malloc(sizeof(*new_location));
 if (new_location == NULL) {
 return LOCATION_STATUS_OUT_OF_MEMORY;
 }

 new_location->id = id;
 memcpy(new_location->name, normalized_name,
 strlen(normalized_name) + 1);
 new_location->next = NULL;

 if (list->tail == NULL) {
 list->head = new_location;
 } else {
 list->tail->next = new_location;
 }
 list->tail = new_location;
 list->count++;

 return LOCATION_STATUS_OK;
}

const Location *location_find_by_id(const LocationList *list, int id)
{
 const Location *current;

 if (list == NULL || id <= 0) {
 return NULL;
 }

 for (current = list->head; current != NULL; current = current->next) {
 if (current->id == id) {
 return current;
 }
 }

 return NULL;
}

const Location *location_find_by_name(const LocationList *list,
 const char *name)
{
 char normalized_name[LOCATION_NAME_CAPACITY];
 const Location *current;

 if (list == NULL || name == NULL ||
 normalize_name(name, normalized_name) != LOCATION_STATUS_OK) {
 return NULL;
 }

 for (current = list->head; current != NULL; current = current->next) {
 if (names_equal_case_insensitive(current->name, normalized_name)) {
 return current;
 }
 }

 return NULL;
}

void location_list_display(const LocationList *list)
{
```

```
 const Location *current;

 if (list == NULL) {
 puts("Location list is unavailable.");
 return;
 }
 if (list->head == NULL) {
 puts("No campus locations have been added.");
 return;
 }

 puts("Campus locations:");
 for (current = list->head; current != NULL; current = current->next) {
 printf(" ID: %d | Name: %s\n", current->id, current->name);
 }
}

void location_list_destroy(LocationList *list)
{
 Location *current;

 if (list == NULL) {
 return;
 }

 current = list->head;
 while (current != NULL) {
 Location *next = current->next;
 free(current);
 current = next;
 }

 location_list_init(list);
}

const char *location_status_message(LocationStatus status)
{
 switch (status) {
 case LOCATION_STATUS_OK:
 return "Location added successfully.";
 case LOCATION_STATUS_INVALID_ARGUMENT:
 return "Invalid argument supplied.";
 case LOCATION_STATUS_INVALID_ID:
 return "Location ID must be a positive integer.";
 case LOCATION_STATUS_EMPTY_NAME:
 return "Location name cannot be empty.";
 case LOCATION_STATUS_NAME_TOO_LONG:
 return "Location name must be at most 63 characters.";
 case LOCATION_STATUS_DUPLICATE_ID:
 return "That location ID is already in use.";
 case LOCATION_STATUS_DUPLICATE_NAME:
 return "That location name is already in use.";
 case LOCATION_STATUS_OUT_OF_MEMORY:
 return "Memory allocation failed; no location was added.";
 default:
 return "Unknown location status.";
 }
}
```

locations.h

```
#ifndef LOCATIONS_H
#define LOCATIONS_H

#include <stddef.h>

#define LOCATION_NAME_MAX 63
#define LOCATION_NAME_CAPACITY (LOCATION_NAME_MAX + 1)

typedef struct Location {
 int id;
 char name[LOCATION_NAME_CAPACITY];
 struct Location *next;
} Location;

typedef struct {
 Location *head;
 Location *tail;
 size_t count;
} LocationList;

typedef enum {
 LOCATION_STATUS_OK = 0,
 LOCATION_STATUS_INVALID_ARGUMENT,
 LOCATION_STATUS_INVALID_ID,
 LOCATION_STATUS_EMPTY_NAME,
 LOCATION_STATUS_NAME_TOO_LONG,
 LOCATION_STATUS_DUPLICATE_ID,
 LOCATION_STATUS_DUPLICATE_NAME,
 LOCATION_STATUS_OUT_OF_MEMORY
} LocationStatus;

void location_list_init(LocationList *list);
LocationStatus location_list_add(LocationList *list, int id, const char *name);
const Location *location_find_by_id(const LocationList *list, int id);
const Location *location_find_by_name(const LocationList *list,
 const char *name);
void location_list_display(const LocationList *list);
void location_list_destroy(LocationList *list);
const char *location_status_message(LocationStatus status);

#endif
```

queue.c

```c
#include "queue.h"

#include <stdint.h>
#include <stdlib.h>

bool queue_init(Queue *queue, size_t capacity)
{
 if (queue == NULL || capacity == 0 ||
 capacity > SIZE_MAX / sizeof(*queue->items)) {
 return false;
 }

 queue->items = NULL;
 queue->capacity = 0;
 queue->front = 0;
 queue->rear = 0;
 queue->count = 0;

 queue->items = malloc(capacity * sizeof(*queue->items));
 if (queue->items == NULL) {
 return false;
 }

 queue->capacity = capacity;
 return true;
}

bool queue_enqueue(Queue *queue, int value)
{
 if (queue == NULL || queue->items == NULL || queue_is_full(queue)) {
 return false;
 }

 queue->items[queue->rear] = value;
 queue->rear = (queue->rear + 1) % queue->capacity;
 queue->count++;
 return true;
}

bool queue_dequeue(Queue *queue, int *value)
{
 if (queue == NULL || value == NULL || queue->items == NULL ||
 queue_is_empty(queue)) {
 return false;
 }

 *value = queue->items[queue->front];
 queue->front = (queue->front + 1) % queue->capacity;
 queue->count--;
 return true;
}

bool queue_is_empty(const Queue *queue)
{
 return queue == NULL || queue->count == 0;
}

bool queue_is_full(const Queue *queue)
{
 return queue != NULL && queue->capacity > 0 &&
 queue->count == queue->capacity;
}

void queue_destroy(Queue *queue)
{
 if (queue == NULL) {
 return;
 }

 free(queue->items);
 queue->items = NULL;
 queue->capacity = 0;
 queue->front = 0;
 queue->rear = 0;
 queue->count = 0;
}
```

queue.h

```h
#ifndef QUEUE_H
#define QUEUE_H

#include <stdbool.h>
#include <stddef.h>

typedef struct {
 int *items;
 size_t capacity;
 size_t front;
 size_t rear;
 size_t count;
} Queue;

/* Initialize an empty queue with storage for capacity values. */
bool queue_init(Queue *queue, size_t capacity);
bool queue_enqueue(Queue *queue, int value);
bool queue_dequeue(Queue *queue, int *value);
bool queue_is_empty(const Queue *queue);
bool queue_is_full(const Queue *queue);
void queue_destroy(Queue *queue);

#endif
```

test_driver.c

```c
#include "locations.h"
#include "graph.h"
#include "queue.h"

#include <limits.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define CHECK(condition) \
 do { \
 if (!(condition)) { \
 fprintf(stderr, "FAIL at %s:%d: %s\n", __FILE__, __LINE__, \
 #condition); \
 return EXIT_FAILURE; \
 } \
 } while (0)

static int test_location_module(void)
{
 LocationList locations;
 const Location *found;
 char exact_length_name[LOCATION_NAME_CAPACITY];
 size_t index;

 location_list_init(&locations);
 CHECK(locations.head == NULL);
 CHECK(locations.tail == NULL);
 CHECK(locations.count == 0);

 CHECK(location_list_add(&locations, 10, "North Gate") ==
 LOCATION_STATUS_OK);
 CHECK(location_list_add(&locations, 73, "Library") ==
 LOCATION_STATUS_OK);
 CHECK(locations.count == 2);

 CHECK(location_list_add(&locations, 10, "New Building") ==
 LOCATION_STATUS_DUPLICATE_ID);
 CHECK(location_list_add(&locations, 420, "lIbRaRy") ==
 LOCATION_STATUS_DUPLICATE_NAME);
 CHECK(location_list_add(&locations, 0, "Invalid ID") ==
 LOCATION_STATUS_INVALID_ID);
 CHECK(location_list_add(&locations, -5, "Negative ID") ==
 LOCATION_STATUS_INVALID_ID);
 CHECK(location_list_add(NULL, 420, "Null list") ==
 LOCATION_STATUS_INVALID_ARGUMENT);
 CHECK(location_list_add(&locations, 420, NULL) ==
 LOCATION_STATUS_INVALID_ARGUMENT);
 CHECK(location_list_add(&locations, 420, " ") ==
 LOCATION_STATUS_EMPTY_NAME);
 CHECK(location_list_add(&locations, 420,
 "This location name is deliberately longer "
 "than sixty-three characters") ==
 LOCATION_STATUS_NAME_TOO_LONG);
 CHECK(locations.count == 2);

 for (index = 0; index < LOCATION_NAME_MAX; index++) {
 exact_length_name[index] = 'X';
 }
 exact_length_name[LOCATION_NAME_MAX] = '\0';
 CHECK(location_list_add(&locations, 420, exact_length_name) ==
 LOCATION_STATUS_OK);
 CHECK(strlen(locations.tail->name) == LOCATION_NAME_MAX);
 CHECK(locations.tail->id == 420);
 CHECK(locations.tail->next == NULL);
 CHECK(locations.count == 3);

 found = location_find_by_id(&locations, 73);
 CHECK(found != NULL);
 CHECK(found->id == 73);
 CHECK(found->name[0] == 'L');
 CHECK(location_find_by_id(&locations, 999) == NULL);

 found = location_find_by_name(&locations, " nOrTh gAtE ");
 CHECK(found != NULL);
 CHECK(found->id == 10);
```

```
 CHECK(location_find_by_name(&locations, "Unlisted Hall") == NULL);

 location_list_destroy(&locations);
 CHECK(locations.head == NULL);
 CHECK(locations.tail == NULL);
 CHECK(locations.count == 0);

 /* Destroying an empty list is safe and leaves it reusable. */
 location_list_destroy(&locations);
 CHECK(location_list_add(&locations, 420, "Auditorium") ==
 LOCATION_STATUS_OK);
 location_list_destroy(&locations);
 CHECK(location_find_by_id(NULL, 10) == NULL);
 CHECK(location_find_by_name(NULL, "North Gate") == NULL);

 return EXIT_SUCCESS;
}

static int test_queue_module(void)
{
 Queue queue = {0};
 int value;

 CHECK(!queue_init(NULL, 3));
 CHECK(queue_is_empty(NULL));
 CHECK(!queue_is_full(NULL));
 CHECK(!queue_enqueue(NULL, 1));
 CHECK(!queue_dequeue(NULL, &value));
 queue_destroy(NULL);

 CHECK(queue_init(&queue, 3));
 CHECK(queue_is_empty(&queue));
 CHECK(!queue_is_full(&queue));
 CHECK(!queue_dequeue(&queue, &value));

 CHECK(queue_enqueue(&queue, 11));
 CHECK(queue_enqueue(&queue, 22));
 CHECK(queue_enqueue(&queue, 33));
 CHECK(queue_is_full(&queue));
 CHECK(!queue_enqueue(&queue, 44));

 CHECK(queue_dequeue(&queue, &value) && value == 11);
 CHECK(queue_enqueue(&queue, 44)); /* Wrap around to the start of the array. */
 CHECK(queue_dequeue(&queue, &value) && value == 22);
 CHECK(queue_dequeue(&queue, &value) && value == 33);
 CHECK(queue_dequeue(&queue, &value) && value == 44);
 CHECK(queue_is_empty(&queue));
 CHECK(!queue_dequeue(&queue, &value));

 /* Repeated single-item cycles exercise wraparound many times. */
 {
 int cycle;
 for (cycle = 0; cycle < 100; cycle++) {
 CHECK(queue_enqueue(&queue, cycle));
 CHECK(queue_dequeue(&queue, &value));
 CHECK(value == cycle);
 }
 }
 CHECK(!queue_dequeue(&queue, NULL));

 queue_destroy(&queue);
 CHECK(queue.items == NULL);
 CHECK(queue.capacity == 0);
 CHECK(!queue_init(&queue, 0));
 CHECK(!queue_init(&queue, SIZE_MAX));
 return EXIT_SUCCESS;
}

static int test_graph_module(void)
{
 Graph graph;
 Graph direct_graph;
 Graph overflow_graph;
 Graph full_graph;
 int route[GRAPH_MAX_VERTICES];
 int reachable[GRAPH_MAX_VERTICES];
 int distance;
 int edge_weight;
 int id;
 size_t route_length;
 size_t reachable_count;
```

```
size_t index;
bool connected;

graph_init(&graph);
CHECK(graph_vertex_count(&graph) == 0);
CHECK(graph_add_vertex(&graph, 10) == GRAPH_STATUS_OK);
CHECK(graph_add_vertex(&graph, 73) == GRAPH_STATUS_OK);
CHECK(graph_add_vertex(&graph, 420) == GRAPH_STATUS_OK);
CHECK(graph_add_vertex(&graph, 501) == GRAPH_STATUS_OK);
CHECK(graph_add_vertex(&graph, 900) == GRAPH_STATUS_OK);
CHECK(graph_add_vertex(&graph, 73) == GRAPH_STATUS_DUPLICATE_VERTEX);
CHECK(graph_add_vertex(&graph, 0) == GRAPH_STATUS_INVALID_ID);
CHECK(graph_add_vertex(&graph, -10) == GRAPH_STATUS_INVALID_ID);
CHECK(graph_add_vertex(NULL, 1) == GRAPH_STATUS_INVALID_ARGUMENT);

CHECK(graph_add_edge(&graph, 10, 73, 120) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 73, 420, 170) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 10, 501, 260) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 501, 420, 80) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 73, 501, 150) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 10, 420, 400) == GRAPH_STATUS_OK);
CHECK(graph_add_edge(&graph, 10, 73, 130) == GRAPH_STATUS_DUPLICATE_EDGE);
CHECK(graph_add_edge(&graph, 10, 10, 100) ==
GRAPH_STATUS_SELF_CONNECTION);
CHECK(graph_add_edge(&graph, 10, 420, 0) ==
GRAPH_STATUS_INVALID_DISTANCE);
CHECK(graph_add_edge(&graph, 10, 420, -1) ==
GRAPH_STATUS_INVALID_DISTANCE);
CHECK(graph_add_edge(&graph, 10, 999, 10) ==
GRAPH_STATUS_VERTEX_NOT_FOUND);
CHECK(graph_get_edge_weight(&graph, 73, 10, &edge_weight));
CHECK(edge_weight == 120); /* An undirected edge works in both directions. */
CHECK(graph_get_edge_weight(&graph, 10, 73, &edge_weight));
CHECK(edge_weight == 120);

CHECK(graph_shortest_path(&graph, 10, 420, route,
GRAPH_MAX_VERTICES, &route_length,
&distance) == GRAPH_STATUS_OK);
CHECK(route_length == 3);
CHECK(route[0] == 10 && route[1] == 73 && route[2] == 420);
CHECK(distance == 290);
/* The 400 m direct edge loses to the 290 m indirect route. */
CHECK(graph_shortest_path(&graph, 10, 420, route, 1, &route_length,
&distance) == GRAPH_STATUS_BUFFER_TOO_SMALL);
CHECK(route_length == 3);
CHECK(graph_shortest_path(&graph, 10, 10, route, GRAPH_MAX_VERTICES,
&route_length, &distance) == GRAPH_STATUS_OK);
CHECK(route_length == 1 && route[0] == 10 && distance == 0);
CHECK(graph_shortest_path(&graph, 10, 900, route, GRAPH_MAX_VERTICES,
&route_length, &distance) == GRAPH_STATUS_NO_ROUTE);
CHECK(route_length == 0 && distance == 0);
CHECK(graph_shortest_path(&graph, 999, 10, route, GRAPH_MAX_VERTICES,
&route_length, &distance) ==
GRAPH_STATUS_VERTEX_NOT_FOUND);
CHECK(graph_shortest_path(&graph, 10, 999, route, GRAPH_MAX_VERTICES,
&route_length, &distance) ==
GRAPH_STATUS_VERTEX_NOT_FOUND);
CHECK(graph_shortest_path(&graph, -1, 10, route, GRAPH_MAX_VERTICES,
&route_length, &distance) ==
GRAPH_STATUS_INVALID_ID);

CHECK(graph_bfs(&graph, 10, reachable, GRAPH_MAX_VERTICES,
&reachable_count) == GRAPH_STATUS_OK);
CHECK(reachable_count == 4);
CHECK(reachable[0] == 10 && reachable[1] == 73 &&
reachable[2] == 420 && reachable[3] == 501);
CHECK(graph_bfs(&graph, 10, reachable, 1, &reachable_count) ==
GRAPH_STATUS_BUFFER_TOO_SMALL);
CHECK(reachable_count == 4);
CHECK(graph_bfs(&graph, 999, reachable, GRAPH_MAX_VERTICES,
&reachable_count) == GRAPH_STATUS_VERTEX_NOT_FOUND);
CHECK(graph_bfs(&graph, -1, reachable, GRAPH_MAX_VERTICES,
&reachable_count) == GRAPH_STATUS_INVALID_ID);
CHECK(graph_bfs(NULL, 10, reachable, GRAPH_MAX_VERTICES,
&reachable_count) == GRAPH_STATUS_INVALID_ARGUMENT);
CHECK(graph_bfs(&graph, 10, NULL, GRAPH_MAX_VERTICES,
&reachable_count) == GRAPH_STATUS_INVALID_ARGUMENT);
CHECK(graph_is_connected(&graph, &connected) == GRAPH_STATUS_OK);
CHECK(!connected);

CHECK(graph_add_edge(&graph, 420, 900, 220) == GRAPH_STATUS_OK);
```

```
    CHECK(graph_is_connected(&graph, &connected) == GRAPH_STATUS_OK);
    CHECK(connected);
    CHECK(graph_bfs(&graph, 900, reachable, GRAPH_MAX_VERTICES,
    &reachable_count) == GRAPH_STATUS_OK);
    CHECK(reachable_count == 5);
    CHECK(reachable[0] == 900);

    CHECK(graph_remove_vertex(&graph, 73) == GRAPH_STATUS_OK);
    CHECK(!graph_has_vertex(&graph, 73));
    CHECK(graph_vertex_count(&graph) == 4);
    CHECK(graph_get_edge_weight(&graph, 10, 420, &edge_weight));
    CHECK(edge_weight == 400);
    CHECK(graph_get_edge_weight(&graph, 10, 501, &edge_weight));
    CHECK(edge_weight == 260);
    CHECK(graph_get_vertex_id_at(&graph, 1, &id));
    CHECK(id == 420);

    graph_init(&overflow_graph);
    CHECK(graph_add_vertex(&overflow_graph, 1) == GRAPH_STATUS_OK);
    CHECK(graph_add_vertex(&overflow_graph, 2) == GRAPH_STATUS_OK);
    CHECK(graph_add_vertex(&overflow_graph, 3) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&overflow_graph, 1, 2, INT_MAX) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&overflow_graph, 2, 3, INT_MAX) == GRAPH_STATUS_OK);
    CHECK(graph_shortest_path(&overflow_graph, 1, 3, route,
    GRAPH_MAX_VERTICES, &route_length,
    &distance) == GRAPH_STATUS_DISTANCE_OVERFLOW);

    graph_init(&full_graph);
    for (index = 0; index < GRAPH_MAX_VERTICES - 1; index++) {
    CHECK(graph_add_vertex(&full_graph, 2000 + (int)index) ==
    GRAPH_STATUS_OK);
    }
    CHECK(graph_vertex_count(&full_graph) == 99);
    CHECK(graph_add_vertex(&full_graph, 9998) == GRAPH_STATUS_OK);
    CHECK(graph_vertex_count(&full_graph) == GRAPH_MAX_VERTICES);
    CHECK(graph_add_vertex(&full_graph, 9999) ==
    GRAPH_STATUS_CAPACITY_REACHED);

    graph_init(&direct_graph);
    CHECK(graph_add_vertex(&direct_graph, 10) == GRAPH_STATUS_OK);
    CHECK(graph_add_vertex(&direct_graph, 73) == GRAPH_STATUS_OK);
    CHECK(graph_add_vertex(&direct_graph, 420) == GRAPH_STATUS_OK);
    CHECK(graph_add_vertex(&direct_graph, 501) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&direct_graph, 10, 420, 100) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&direct_graph, 10, 73, 30) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&direct_graph, 73, 420, 100) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&direct_graph, 10, 501, 40) == GRAPH_STATUS_OK);
    CHECK(graph_add_edge(&direct_graph, 501, 420, 100) == GRAPH_STATUS_OK);
    CHECK(graph_shortest_path(&direct_graph, 10, 420, route,
    GRAPH_MAX_VERTICES, &route_length,
    &distance) == GRAPH_STATUS_OK);
    CHECK(route_length == 2 && route[0] == 10 && route[1] == 420);
    CHECK(distance == 100);

    graph_init(&graph);
    CHECK(graph_is_connected(&graph, &connected) == GRAPH_STATUS_OK);
    CHECK(!connected);
    CHECK(graph_add_vertex(&graph, 42) == GRAPH_STATUS_OK);
    CHECK(graph_is_connected(&graph, &connected) == GRAPH_STATUS_OK);
    CHECK(connected);

    return EXIT_SUCCESS;
}

int main(void)
{
    if (test_location_module() != EXIT_SUCCESS) {
    return EXIT_FAILURE;
    }
    if (test_queue_module() != EXIT_SUCCESS) {
    return EXIT_FAILURE;
    }
    if (test_graph_module() != EXIT_SUCCESS) {
    return EXIT_FAILURE;
    }

    puts("Location, queue, and graph module tests passed.");
    return EXIT_SUCCESS;
}
```